Low-Latency Pipelined RISC-V 32-bit Processor

Dr.Nayana D K
Electronics and Communication
Reva University
Bangalore, India
nayanadk@reva.edu.in

Manjunath S R
Electronics and Communication
Reva University
Bangalore, India
ugcet220427@reva.edu.in

Mithun Nagaraja
Electronics and Communication
Reva University
Bangalore, India
ugcet2210250@Reva.edu.in

Darshan N A
Electronics and Communication
Reva University
Bangalore, India
22020133864@reva.edu.in

Nagachandra I Dodwad
Electronics and Communication
Reva University
Bangalore, India
ugcet22059@reva.edu.in

ABSTRACT

This paper describes the design of a 32-bit RISC-V processor that uses a five-stage pipeline to reduce execution delay compared to basic single-cycle and non-pipelined CPUs. The proposed core supports the RV32I instruction set and divides instruction execution into Instruction Fetch, Decode, Execute, Memory Access, and Write-Back stages so that several instructions stay in the pipeline at the same time. To handle hazards, the design includes a forwarding unit and a hazard detection block that insert stalls only when they are really needed while keeping the operation correct. The processor is written in Verilog HDL and is planned for FPGA synthesis and simulation, where Cycles Per Instruction, latency, and resource usage will be compared with a non-pipelined design. Because RISC-V is modular and open, the core is suitable for embedded and real-time applications and can be extended for future work in custom instruction design.

Keywords

RISC-V, RV32I, pipelined processor, low latency, hazard detection, forwarding, FPGA.

I. INTRODUCTION

RISC-V is a modern, open instruction set architecture that was created to support teaching, research, and commercial processor development without the license and royalty barriers of traditional proprietary ISAs. Because the base instruction set is small and clean, it is easier for students and designers to understand how each instruction is encoded and executed in hardware, while optional extensions allow the same ISA family to scale from tiny microcontrollers to high-performance processors. This combination of openness and modularity has made RISC-V a popular choice for academic projects that aim to explore real processor design rather than only theoretical models.

In a simple single-cycle CPU, all steps of an instruction-fetch, decode, execute, memory access, and write-back are completed within one long clock period.

The clock must be slow enough to accommodate the worst-case instruction path, so the overall throughput is low even when most instructions are simpler. Multi-cycle designs improve the maximum frequency by spreading work across several cycles, but they still allow only one instruction to move through the datapath at a time, which limits how many instructions can complete per unit time. These limitations become more visible in applications that require quick responses or must handle many operations in parallel, such as embedded control or real-time systems.

Pipelining addresses this problem by dividing instruction execution into a sequence of stages and allowing different instructions to occupy different stages simultaneously, similar to an assembly line. The classic five-stage RISC pipeline, consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB), offers a good balance between performance gain and hardware complexity for 32-bit integer processors. When the pipeline runs smoothly, a new instruction can complete on almost every clock cycle, giving much higher throughput than a single-cycle core that uses the same functional units.

However, a pipelined design also introduces new challenges. Data hazards occur when an instruction needs a value that is still being computed by an earlier instruction, and structural hazards arise if multiple stages compete for the same hardware resource. Control hazards appear around branch and jump instructions, where the next program counter value is not known early enough, potentially causing the pipeline to fetch the wrong instructions. If these hazards are not handled properly, the processor may either produce incorrect results or lose much of the expected speedup due to frequent stalls and flushes.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

The project described in this paper focuses on designing a low-latency 32-bit RISC-V processor that implements the RV32I base instruction set using a five-stage pipelined microarchitecture with clear and educational hazard-handling mechanisms. The main goals are to increase instruction throughput compared to a non-pipelined baseline, to keep the datapath and control logic understandable for learning purposes, and to target FPGA implementation so that the core can be tested in real hardware. By integrating data forwarding, a simple hazard detection unit, and basic control-hazard management, the design aims to demonstrate how theoretical concepts from computer architecture courses translate into a working pipelined CPU suitable for embedded and real-time applications.

II. LITERATURE SURVEY

[1] A. Waterman and K. Asanović presented one of the earliest complete descriptions of the RISC-V instruction set, defining the RV32I base ISA and explaining its simplicity, modular extensions, and suitability for both academic and industrial processor designs. Their work established the architectural foundation that many later RV32I and RV32IM pipeline implementations follow, and it is widely used as the primary reference for encoding, privilege levels, and software toolchain support.

[2] D. Harris and S. Harris discussed the implementation of five-stage pipelined RISC-V processors in a teaching-oriented context, focusing on clean datapath and control partitioning, hazard handling, and FPGA realization. Their book details the design of a basic RV32I pipeline with forwarding and hazard detection units, providing reference Verilog structures that influenced several educational and prototype cores.

[3] H. Miyazaki, T. Kanamori, M. A. Islam, and K. Kise introduced RVCoreP, an optimized five-stage RV32I soft processor targeting FPGA platforms and emphasizing higher operating frequency. The authors improved the instruction-fetch path, ALU timing, and data-alignment logic, and reported better clock speeds and resource usage compared with naiver RV32I implementations, showing how careful microarchitectural tuning can benefit pipelined RISC-V cores.

[4] H. Li, C. Jing, and J. Liu proposed the NRP processor, a performance-optimised RV32I. five-stage pipeline that refines the decode stage and introduces a simple branch-handling scheme to cut control-hazard penalties. Their results demonstrate noticeable reductions in CPI and improved throughput on FPGA without a large increase in hardware complexity, motivating the use of lightweight prediction and more accurate control logic in student-level designs.

[5] Recent low-power 32-bit RISC-V cores for edge and IoT computing have combined five-stage pipelines with clock-gating and power-gating strategies to lower dynamic energy while keeping latency small. These designs show that careful control of switching activity, simplified datapaths, and compact hazard units can produce RV32I processors that meet both performance and energy constraints in real-time embedded deployments.

Educational projects and open-source RISC-V soft cores published by universities and community groups demonstrate minimal RV32I pipelines on FPGAs, often focusing on clarity of Verilog structure rather than peak performance. Such works typically implement basic forwarding and stall logic, provide simple test programs, and serve as reference designs that help students understand how instruction set specifications map to a working pipelined microarchitecture.

III. EASE OF USE

The proposed low-latency RISC-V processor is intentionally designed to be simple to learn, implement, and extend, so that it can be used comfortably by students and engineers with basic HDL experience. The following features contribute to the overall ease of use of the system.

- **Modular Verilog Structure:**

The complete processor is split into small, logically independent Verilog modules such as instruction-memory interface, register file, ALU, control unit, forwarding unit, hazard detection block, and individual pipeline registers. Each module has a clearly defined set of inputs and outputs, which allows users to open, simulate, or modify a single block without being forced to understand the entire design at once.

- **Textbook-Style Five-Stage Pipeline:**

The datapath strictly follows the classical five-stage pipeline (IF, ID, EX, MEM, WB) presented in standard computer architecture textbooks and training material. Because the stage boundaries are obvious and named in the same way, learners can directly relate lecture diagrams and timing examples to the implemented hardware, reducing the learning curve when moving from theory to RTL.

- **Transparent Hazard Handling:**

Data hazards are managed using simple comparator logic that checks register numbers and selects forwarded values, while load-use hazards trigger a single-cycle stall generated

by the hazard detection unit. The corresponding control signals (stall, forward, flush) are observable in simulation waveforms, making it straightforward for users to trace why a particular instruction is delayed or why certain pipeline registers are cleared.

- **FPGA-Friendly Implementation Flow:**

The HDL code is written using synthesizable constructs compatible with common FPGA tools such as Xilinx Vivado, avoiding vendor-specific primitives except for memory blocks. Timing is assisted by explicit pipeline registers between stages, which helps new users obtain successful synthesis and timing closure without complex constraints, and resource reports (LUTs, FFs, BRAMs) give quick feedback on hardware cost.

- **Clear Test and Debug Strategy:**

The project includes simple RISC-V test programs that exercise arithmetic, branch, and load/store instructions, so users can run step-by-step simulations and see how instructions move through each stage. Because the design exposes key internal signals such as PC, instruction word, register values, and ALU results, it becomes easier to debug functional errors or verify new extensions.

- **Extensibility for Student Projects:**

Since the datapath and control logic are separated and the RV32I subset is well-defined, users can add new instructions, change the branch policy, or attach small instruction/data caches without rewriting the existing core. This makes the processor a convenient starting point for mini-projects, internships, or further research on topics like branch prediction, security extensions, or accelerator integration.

IV. METHODOLOGY

The project follows a step-by-step approach starting from ISA study and ending with FPGA-oriented verification.

A. Study of RV32I

First, the RV32I base instruction set is studied in detail, including the six instruction formats (R, I, S, B, U, J), opcode and function fields, and register conventions. From this, the required control signals for ALU operations, immediate generation, memory access, and branching are listed so that later stages of the design can be planned.

B. Pipeline Stage Design

The processor uses five pipeline stages.

- Instruction Fetch (IF): Reads the next instruction from instruction memory using the current program counter and calculates the next PC, which may be sequential or a branch target.
- Instruction Decode (ID): Decodes the instruction fields, reads source operands from the register file, and generates control signals for the following stages.
- Execute (EX): Uses the ALU to perform arithmetic or logical operations and to compute branch conditions and target addresses.
- Memory Access (MEM): Performs load and store operations by interacting with data memory, while non-memory instructions simply pass their ALU results forward.
- Write-Back (WB): Writes the final result, either from the ALU or from memory, back into the destination register.

Pipeline registers are placed between each pair of stages to store intermediate values and control bits so that multiple instructions can be in different stages at the same time.

C. Hazard Handling

To keep the pipeline correct and efficient, three kinds of hazards are considered.

Data Hazards: These occur when an instruction needs a result that is still in the pipeline. A forwarding unit compares source and destination register numbers and, when necessary, forwards results from EX, MEM, or WB directly to the EX stage inputs instead of waiting for normal write-back.

Load-Use Hazards: If an instruction uses data loaded from memory in the previous cycle, forwarding alone is not enough because the data is available only after MEM. A hazard detection unit then inserts a one-cycle stall by freezing IF and ID and inserting a bubble between ID and EX.

Control Hazards: For branch instructions, the pipeline assumes sequential flow and later corrects it if the branch is taken. When a misprediction is detected, the instructions that were fetched wrongly are flushed from the pipeline, and the PC is updated to the correct target.

D. Implementation and Verification

The design is written in Verilog HDL using a modular hardware description, where each stage, the register file, ALU, and hazard units are separate modules with clear interfaces. Simulation is planned in tools such as Xilinx Vivado or ModelSim, using small assembly programs that test arithmetic, branching, memory access, and hazard situations. Performance will be studied by measuring CPI, latency for

different instruction types, and FPGA resource usage such as LUTs, flip-flops, and on-chip memory blocks, and these results will be compared with a simpler non-pipelined core.

V. Design Architecture

A. Block Diagram

The overall architecture consists of instruction memory, a program counter, register file, ALU, data memory, control unit, forwarding unit, and hazard detection logic connected through the five pipeline stages. The program counter and instruction memory belong mainly to the IF stage, the register file and control logic to ID, the ALU and branch unit to EX, data memory to MEM, and the write-back multiplexers to WB.

Pipeline registers labeled IF/ID, ID/EX, EX/MEM, and MEM/WB store instruction fields, operand values, and control signals between stages so that data flows smoothly through the pipeline. Forwarding paths run from EX/MEM and MEM/WB back to the ALU inputs, and control signals for stall and flush modify the normal pipeline behavior when hazards are detected.

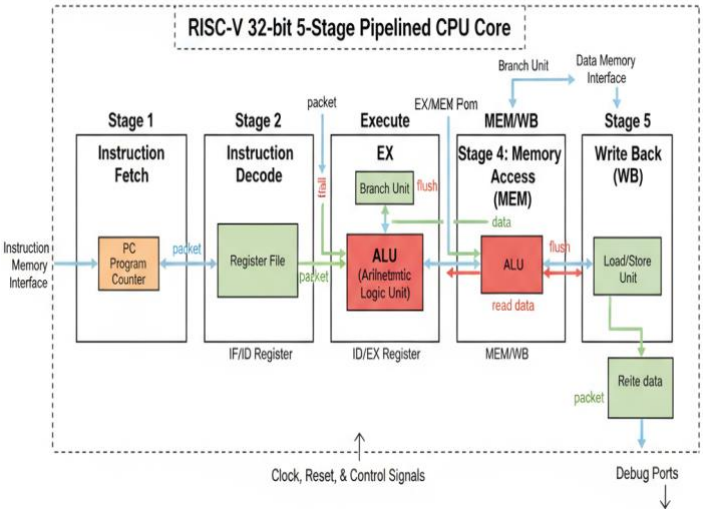


Fig. 1. *Block diagram of the RISC-V 32-bit five-stage pipelined CPU core*

B. Datapath and Control Signals

The datapath is 32 bits wide and supports operations defined by RV32I, including arithmetic, logical, shift, branch, and load/store instructions. Basic control signals such as RegWrite, MemRead, MemWrite, MemToReg, ALUSrc, and Branch are generated in the ID stage and passed through the pipeline registers so that each stage knows what operation to perform.

The hazard detection and forwarding units adjust these control signals when dependencies are present, temporarily stopping the PC and IF/ID register for stalls or clearing certain pipeline registers when a flush is required. This organization keeps the control logic understandable while still handling the main hazard cases seen in a basic pipelined core.

VI. Simulation

The testbench applies 2 reset cycles then runs for 60 clock cycles (600 ns at 100 MHz). It automatically checks all register and memory values and prints PASS or FAIL.

A. TCI Console Output

```
=== RESET RELEASED - Simulation START ===

========== REGISTER FILE CHECK ==========

X1 = 10 (expected 10) PASS   x2 = 20

(expected 20) PASS   x3 = 30 (expected

30) PASS   x4 = -10 (expected -10) PASS

X5 = 0 (expected 0) PASS   x6 = 30

(expected 30) PASS   x7 = 30 (expected

 30) PASS   x8 = 30 (expected 30) PASS

X9 = 60 (expected 60) PASS   x10 = 0

(expected 0) PASS   x11 = 77 (expected

77) PASS

========== DATA MEMORY CHECK ==========

dmem[0] = 30 (expected 30) PASS

========== FORWARDING UNIT CHECK ==========

EX->EX forward test (x3 = x1 + x2): PASS

MEM->EX forward test (x9 = x8 + X3): PASS

========== BRANCH FLUSH CHECK ==========

X10 should be 0(addi x10, x0, 99 was
skipped): PASS

========== SIMULATION COMPLETE ==========
```

B. What Each Result Proves

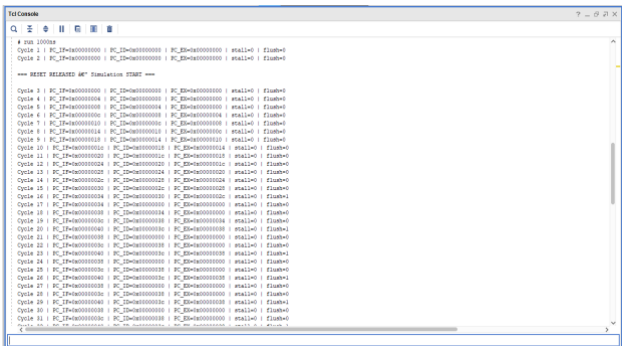
Check	What It Proves
x1 = 10, x2 = 20	I-type ADDI instruction works correctly
x3 = 30 (ADD)	R-type ADD works; EX-to-EX forwarding supplies correct operands
x4 = -10 (SUB)	R-type SUB works; signed 32-bit arithmetic correct
x5 = 0 (AND)	Bitwise AND: 01010 AND 10100 = 00000 = 0
x6 = 30 (OR)	Bitwise OR: 01010 OR 10100 = 11110 = 30
x7 = 30 (XOR)	Bitwise XOR: 01010 XOR 10100 = 11110 = 30
x8 = 30 (LW)	SW stored 30 to mem[0]; LW loaded it back correctly
x9 = 60 (ADD)	MEM-to-EX forwarding delivered loaded value x8 to the ALU
x10 = 0 (skipped)	BEQ branch was taken; pipeline flush discarded the addi at 0x30
x11 = 77	PC correctly jumped to branch target 0x34; addi x11,x0,77 executed

dmem[0] = 30	SW instruction stored the correct value to data memory

VII. RESULT ANALYSIS

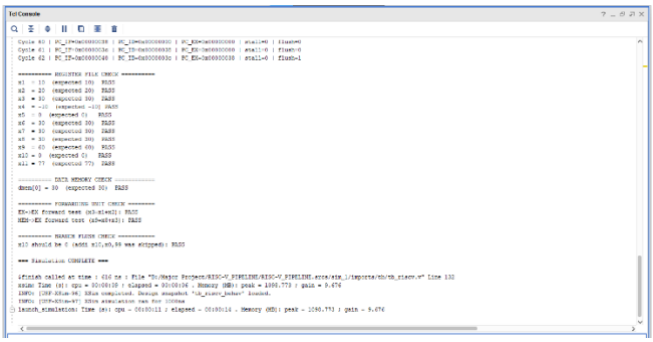
1. Output 1:-

The cycle-by-cycle simulation output of the pipelined RISC-V processor in the Vivado Tcl console. Each line represents a clock cycle and displays the program counter values at different pipeline stages such as Instruction Fetch (PC_IF), Instruction Decode (PC_ID), and Execute (PC_EX). The signals stall and flush indicate pipeline control actions; stall pauses execution during hazards, while flush clears incorrect instructions during branch operations. In the shown cycle, flush = 1, meaning a branch instruction caused the pipeline to remove wrongly fetched instructions. This output verifies that the pipeline execution and hazard handling are functioning correctly.



2. Output 2:-

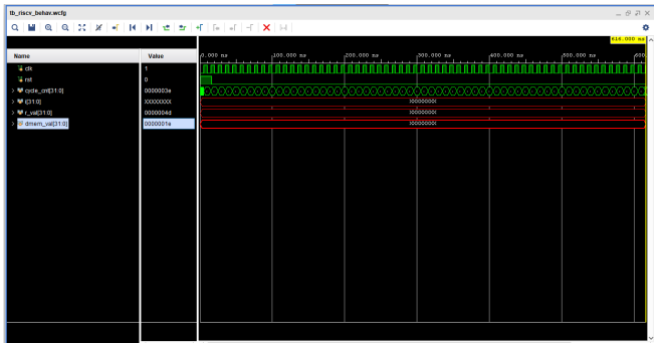
The final Tcl console output of the RISC-V pipeline simulation after executing the test program. It verifies the register file values, where registers like x1=10, x2=20, x3=30, x4=-10, x9=60, and x11=77 match the expected results and all tests show PASS. The data memory check also confirms the correct value stored in memory. Additionally, the forwarding unit tests and branch flush check pass successfully, proving that hazard handling works correctly. Finally, the message "Simulation COMPLETE" indicates that the processor executed the program correctly without errors.



WAVEFORMS:-

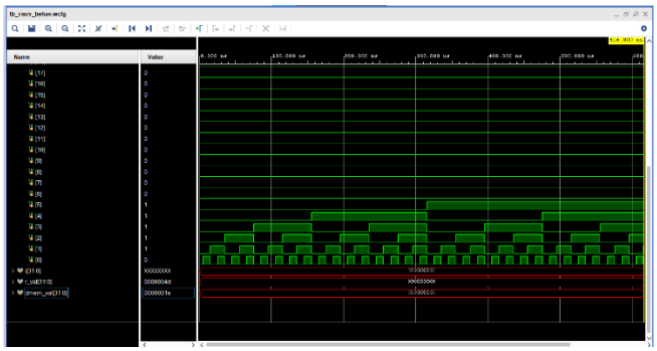
1. Output 1:-

the waveform output of the RISC-V pipeline simulation in Vivado. It displays signals such as clock (clk), reset (rst), cycle counter, instruction register, register value (r_val), and data memory value (dnem_val) over time. The clock signal toggles continuously, indicating the processor is running through multiple cycles. The waveform helps visualize how signals change during execution and verifies the correct functioning of the processor during simulation. Overall, it confirms that the pipeline operations and memory/register activities are occurring correctly across clock cycles.



2. Output 2:-

The waveform represents the simulation of a 5-stage pipelined RISC-V processor in Xilinx Vivado over approximately 616 ns. Initially, some signals appear undefined (shown in red) due to reset conditions, but they stabilize as the pipeline begins execution. The register values update sequentially, demonstrating correct instruction processing and ALU operations. The final outputs, with r_val = 77 and dmem_val = 30, match the expected results of the test program. Additionally, stable signals and no unexpected changes confirm that forwarding, stalling, and branch flushing mechanisms are functioning correctly.



VIII. Application

The designed processor can be used in different application domains.

- **Embedded Controllers:** The core can serve as a simple controller for industrial automation, small robots, or consumer devices, where quick response and predictable execution are important.
- **Real-Time Tasks:** Systems such as drones, medical monitoring units, and automotive modules require deterministic timing, and a pipelined RISC-V CPU with low latency fits well into such applications.
- **FPGA-Based Systems:** As a soft core, the processor can be integrated into FPGA designs along with accelerators for DSP, AI, or security, benefiting from the open RISC-V ecosystem and allowing custom extensions.
- **Education and Research:** Because the pipeline and hazard units are clearly separated, the design is suitable for teaching the steps from ISA to microarchitecture and for experimenting with additions like branch prediction or caches.

IX. Project Status

This work corresponds to the Phase-2 stage of a major project in the School of ECE at REVA University. In this phase, the topic has been finalized, the RV32I ISA and five-stage pipeline concept have been studied, and a detailed methodology and block-level architecture have been documented.

The next steps include writing the full Verilog RTL for all modules, running functional simulations, measuring CPI and resource usage, and finally synthesizing the design to check timing and real-time behavior. The experience gained from these steps will give the project team practical understanding of how theory from computer architecture and digital design courses is applied in an actual CPU implementation.

X. Conclusion

A low-latency 32-bit RISC-V processor using a five-stage pipeline has been proposed and planned in this work as a student-level yet realistic CPU design. The design targets the RV32I base ISA and focuses on simple but effective mechanisms for data forwarding, hazard detection, and control-hazard handling, which are essential for practical pipelined execution.

The architecture and methodology are suited for FPGA implementation and for performance comparison against simpler non-pipelined cores using CPI, latency, and resource-usage metrics. In future phases, the design can be extended with features such as branch prediction, basic caches, low-power modes, or custom instructions, turning this project into a platform for further research and learning in processor design

Acknowledgment

We thank our guide, Dr. Nayana D K, School of ECE, REVA University, for continuous support, technical discussion, and regular feedback throughout the progress of this work. The authors are also grateful to Dr. K M Sudarshan, Director, School of ECE, and to the management of REVA University for providing laboratory facilities and an

encouraging academic environment. Special thanks are due to the teaching staff, classmates, and family members for their motivation and help during the completion of this phase of the project.

REFERENCES

[1] D. A. Patterson and A. Waterman, The RISC-V Reader: An Open Architecture Atlas, 1st ed., Strawberry Canyon LLC, 2017.

[2] A. Waterman and K. Asanova, The RISC-V Instruction Set Manual – Volume I: User-Level ISA, RISC-V Foundation, 2019.

[3] D. Harris and S. Harris, Digital Design and Computer Architecture – RISC-V Edition, Morgan Kaufmann, 2021.

[4] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed., Morgan Kaufmann, 2019.

[5] H. Miyazaki, T. Kanamori, M. A. Islam, and K. Kise, "RVCoreP: An optimized RISC-V soft processor of five-stage pipelining," 2020.

[6] H. Li, C. Jing, and J. Liu, "Performance-Optimised Design of the RISC-V Five-Stage Pipelined Processor NRP," 2024

[7] F. Hussain, "Design and FPGA Implementation of Five Stage Pipelined RISC-V Processor," *Proc. Int. Conf. Information, Communication and Technology (I2CT)*, 2024. This paper presents a five-stage RV32 pipeline implemented on FPGA and compares maximum clock frequency and power dissipation with a single-cycle core.

[8] K. Miyazaki *et al.*, "RVCoreP: An Optimized RISC-V Soft Processor of Five-Stage Pipelining," *IEICE Trans. Information and Systems*, vol. E103-D, no. 12, pp. 1–9, Nov. 2020. The authors propose RVCoreP, a five-stage RV32I soft core with optimized instruction-fetch, ALU, and data-memory paths, achieving about 30% better performance than VexRiscv on FPGA.

[9] N. Sharma and S. Gupta, "Design and Implementation of a Low-Power RISC-V Processor Core for Edge Devices," *J. Intelligent VLSI and Communication Technologies (JIVCT)*, vol. 15, no. 2, pp. 1–8, Oct. 2025. This work targets energy-constrained IoT nodes using a five-stage RISC-V pipeline with clock-gating and power-gating techniques to reduce dynamic and leakage power while maintaining acceptable throughput.

[10] Y. Chang *et al.*, "Configurable Five-Stage RV32IM Microprocessor for Edge Computing," *Harbin Engineering University Journal*, 2023. The paper introduces a configurable RV32IM core that can operate in low-power RV32I mode or full RV32IM mode, illustrating how a five-stage pipeline can be tuned for either energy efficiency or higher performance in edge-computing scenarios.